Edad del Perro — Pruebas de seguridad

OWASP MASVS · Análisis estático · Vulnerabilidades y re-pruebas · Kali Linux / BadStore

Sarah Isabella Peña · Juan Carlos Diaz · Sebastian Grande Moreno

Docente: Ing. Jairo Armando Salcedo Aranda

Universidad de San Buenaventura · Optativa II: Desarrollo de Aplicaciones para Dispositivos Móviles · Noviembre de 2026

Tabla de contenido

Objetivo y alcance 3
Equipo 3
Herramientas 4
Análisis estático: antes y después 5
Vulnerabilidades encontradas y acciones correctivas 6
Hallazgos informativos aceptados 7
Verificaciones por categoría MASVS 8
Validación de datos de entrada 8
Permisos solicitados 9
Comunicaciones HTTPS 9
Almacenamiento local (adb) 9
Prácticas con Kali Linux y BadStore (laboratorio) 10
Re-pruebas después de las correcciones 11
Conclusiones 12

Objetivo y alcance

Este documento registra las pruebas de seguridad de **Edad del Perro** (paquete com.optativa.dogage) según OWASP MASVS v2 (*Mobile Application Security Verification Standard*) y la guía OWASP MASTG.

Las pruebas se hicieron solo sobre la aplicación del equipo, en emulador o dispositivo propio. Las prácticas con Kali Linux se hacen contra **BadStore**, una tienda vulnerable a propósito para laboratorio.

Equipo

Integrante	Rol Scrum
Sarah Isabella Peña	Product Owner
Juan Carlos Diaz	Scrum Master
Sebastian Grande Moreno	Developer

Herramientas

Herramienta	Tipo	Uso
mobsfscan v1.1.0 (reglas de MobSF)	Análisis estático (SAST)	Código Kotlin y AndroidManifest.xml contra reglas OWASP MASVS.
detect-secrets v1.5.0	Búsqueda de secretos	Claves, tokens y contraseñas en todo el repositorio.
JUnit 4 (41 pruebas)	Pruebas unitarias	Validación de entradas, controles del agente de IA y reglas de negocio.
Android Lint (Android Studio)	Análisis estático	**Code → Inspect Code** y ./gradlew lint.
Kali Linux: Burp Suite / OWASP ZAP	Proxy de interceptación	Verificar que el tráfico de la app es HTTPS.
Kali Linux: nmap, nikto, sqlmap	Escaneo y explotación (laboratorio)	Práctica contra BadStore.
adb (Android Debug Bridge)	Inspección del dispositivo	Revisar el almacenamiento local y los permisos.

Análisis estático: antes y después

Se ejecutó mobsfscan sobre el código antes de las correcciones (versión anterior en GitHub) y después. Los reportes completos están en esta carpeta (mobsfscan_antes.json y mobsfscan_despues.json).

Regla	Severidad	MASVS	Antes	Después
android_task_hijacking2	Alta	platform-1	Sí	**No (corregido)**
android_kotlin_hardcoded	Media	MSTG-STORAGE-14	Sí	Sí
android_manifest_allow_backup	Media	storage-8	Sí	**No (corregido)**
android_certificate_transparency	Informativa	MSTG-NETWORK-4	Sí	Sí
android_prevent_screenshot	Informativa	MSTG-STORAGE-9	Sí	Sí
android_root_detection	Informativa	MSTG-RESILIENCE-1	Sí	Sí
android_safetynet	Informativa	MSTG-RESILIENCE-1	Sí	Sí
android_ssl_pinning	Informativa	MSTG-NETWORK-4	Sí	Sí
android_tapjacking	Informativa	MSTG-PLATFORM-9	Sí	Sí

Comandos para repetir el análisis:

```
pip install mobsfscan detect-secrets
mobsfscan app/
detect-secrets scan --all-files
```

Vulnerabilidades encontradas y acciones correctivas

ID	Vulnerabilidad	MASVS	Riesgo	Acción correctiva	Estado tras re-prueba
V-01	android:allowBackup="true": los datos de la app se podían extraer con adb backup o copiarse en respaldos.	MASVS-STORAGE	Medio	allowBackup="false", fullBackupContent="false" y data_extraction_rules.xml que excluye todos los datos (Android 12+).	✅ Corregido: mobsfscan ya no lo reporta
V-02	StrandHogg 2.0 (secuestro de tareas): otra app podía ponerse encima de la actividad para hacer *phishing* en Android 8 y 9 (minSdk 26).	MASVS-PLATFORM	Alto (herramienta) / Medio (real)	android:taskAffinity="" y android:launchMode="singleInstance" en MainActivity.	✅ Corregido: mobsfscan ya no lo reporta
V-03	El tráfico HTTP en claro no estaba bloqueado explícitamente.	MASVS-NETWORK	Bajo	android:usesCleartextTraffic="false".	✅ Corregido
V-04	Riesgo de exponer la clave del servicio de IA en el código fuente.	MASVS-STORAGE / CODE	Alto	La clave se lee de local.properties (en .gitignore) y llega por BuildConfig; viaja en encabezado HTTPS. Riesgo residual: la clave queda dentro del APK. Se recomienda restringirla en Google Cloud a este paquete + SHA-1.	✅ Sin secretos en el repositorio (detect-secrets)
V-05	Manipulación del agente de IA (*prompt injection*), mensajes enormes y abuso del servicio.	MASVS-CODE	Medio	AiInputGuard (límite de 500 caracteres, filtro de manipulación), AiResponseGuard, límite de 10 mensajes/min, instrucciones del	✅ 13 pruebas automatizadas en verde

ID	Vulnerabilidad	MASVS	Riesgo	Acción correctiva	Estado tras re-prueba
				sistema restringidas.	
FP-01	android_kotlin_hardcoded en DogStore.kt: la constante KEY = "dogs_v1" es el **nombre** de una preferencia, no una credencial.	MASVS-STORAGE	Ninguno	Falso positivo.	Descartado

Hallazgos informativos aceptados

Regla	Decisión
SSL pinning / Certificate Transparency	Aceptado. Se usa la validación TLS estándar de Android. El pinning con Google o Firebase puede romper la app cuando rotan certificados.
Detección de root / SafetyNet (Play Integrity)	Aceptado. App académica sin pagos ni datos críticos.
Bloqueo de capturas de pantalla	Aceptado. No se muestran datos sensibles.
Tapjacking	Aceptado. Riesgo bajo: las acciones destructivas piden confirmación.

Verificaciones por categoría MASVS

Categoría	Verificación	Resultado
MASVS-STORAGE	Datos guardados: dogs_v1: nombre, tamaño y edad de cada perro (no sensible).	✅ No se guardan contraseñas, tokens ni datos personales.
MASVS-STORAGE	Respaldos deshabilitados	✅ V-01 corregido
MASVS-CRYPTO	No se implementa criptografía propia; TLS del sistema	✅
MASVS-AUTH	Sin cuentas de usuario	N/A
MASVS-NETWORK	Todas las comunicaciones por HTTPS; HTTP en claro bloqueado	✅
MASVS-PLATFORM	Permisos mínimos; una sola actividad exportada; protección StrandHogg	✅
MASVS-CODE	Validación de todas las entradas del usuario y del agente de IA	✅
MASVS-RESILIENCE	Sin ofuscación ni detección de root	Aceptado (app académica)
MASVS-PRIVACY	Aviso de IA y recomendación de no compartir datos sensibles en el chat	✅

Validación de datos de entrada

Entrada	Validación
Años / meses	Solo enteros, meses de 0 a 11, años ≤ 30, edad > 0 (DogAgeCalculator.parseAge).
Nombre del perro	2–20 letras, sin números ni símbolos, sin repetir (DogValidator).
Cantidad de perros	Máximo 20.
Chat con IA	Vacío, más de 500 caracteres, caracteres de control y frases de manipulación (AiInputGuard).

Firestore y SharedPreferences no usan SQL, así que no hay inyección SQL. Los textos se muestran como texto plano en Compose (no HTML), así que no hay XSS.

Permisos solicitados

Permiso	Justificación
android.permission.INTERNET	Únicamente para el agente de IA (Gemini). Sin clave, la app funciona sin internet.

La app no pide cámara, ubicación, contactos ni almacenamiento externo.

Verificación en el emulador: Ajustes → Apps → Edad del Perro → Permisos debe mostrar «No se solicitaron permisos» (INTERNET es un permiso normal y no aparece ahí). En consola: `adb shell dumpsys package com.optativa.dogage | grep permission`.

Comunicaciones HTTPS

Servicio	Endpoint	Protocolo
Google Gemini	generativelanguage.googleapis.com	HTTPS (TLS)

Verificación con Kali Linux (Burp Suite):

1. Configurar el proxy del emulador hacia Burp (Kali, puerto 8080).
2. Usar la app. Sin instalar el certificado de Burp, las conexiones deben **fallar** (la app valida TLS).
3. Ninguna petición debe aparecer en HTTP plano.

Almacenamiento local (adb)

```
adb shell run-as com.optativa.dogage ls shared_prefs
adb shell run-as com.optativa.dogage cat shared_prefs/*.xml
adb backup com.optativa.dogage      # debe generar un respaldo vacío (allowBackup=false)
```

Prácticas con Kali Linux y BadStore (laboratorio)

BadStore es una tienda web **vulnerable a propósito**, pensada para practicar. Se ejecuta en una máquina virtual aislada (red solo-anfitrión) junto a Kali Linux. Cada hallazgo se compara con el control que tiene nuestra app.

ID	Prueba en BadStore (Kali)	Hallazgo esperado en BadStore	Control en Edad del Perro	Evidencia (captura)
K-01	nmap -sV <ip_badstore>	Puertos y servicios expuestos (HTTP 80, MySQL 3306)	No exponemos servicios propios; solo se consumen APIs de Google por HTTPS	☐
K-02	nikto -h http://<ip_badstore>	Archivos sensibles, cabeceras inseguras, HTTP sin TLS	Tráfico solo HTTPS (V-03)	☐
K-03	Inyección SQL en el login (' OR '1'='1) / sqlmap	Acceso sin contraseña y volcado de usuarios	Sin login ni SQL; entradas validadas	☐
K-04	XSS en el libro de visitas (<script>alert(1)</script>)	El script se ejecuta en el navegador	Compose muestra texto plano; el chat filtra entradas	☐
K-05	Manipular el precio en el carrito (Burp)	Compra con precio alterado	N/A: no hay compras	☐
K-06	Cookies/sesión sin HttpOnly ni Secure	Robo de sesión	N/A: no hay sesión	☐

Re-pruebas después de las correcciones

ID	Re-prueba	Resultado
V-01, V-02	mobsfscan app/ sobre el código corregido	✅ Las reglas android_manifest_allow_backup y android_task_hijacking2 ya no aparecen
V-03	Revisión del manifiesto y proxy Burp	✅ usesCleartextTraffic="false"
V-04	detect-secrets scan --all-files	✅ Sin claves de IA en el repositorio
V-05	./gradlew testDebugUnitTest (AiCoreTest)	✅ 13/13 pruebas en verde
Todas	Suite completa de pruebas unitarias	✅ 41/41 en verde

Conclusiones

- El análisis estático encontró dos hallazgos reales en el manifiesto (respaldo de datos y StrandHogg 2.0). Se corrigieron y se confirmó con una nueva ejecución.
- La integración del agente de IA se diseñó con la clave fuera del código, validación de entradas y salidas, límite de uso y respuesta local ante cualquier fallo.
- Al no tener servidor ni cuentas, la superficie de ataque es pequeña: los riesgos se concentran en el almacenamiento local y en la clave de IA, ambos controlados.